

Systems Analysis and Design

Chapter 10: MVC model



Learning Objectives

10.1 Definition

10.2 Different understanding of MVC

10.3 Types of MVC

10.4 Example of MVC in different technology

Definition (1 of 2)

- MVC stands for Model-View-Controller (or 3-tier architecture)
- The Model is the actual internal representation
- The View (or a View) is a way of looking at or displaying the model
- The Controller provides for user input and modification
- These three components are usually implemented as separate classes

Definition (1 of 2)

- MVC (or 3-tier) is known as:
 - Architecture
 - Pattern
 - Or Architectural Pattern
- Trygve Reenskaug invented MVC
- MVC were written when he was visiting a scientist at Xerox Palo Alto Research Laboratory (PARC) in 1978/79
- At first, MVC was called “Thing Model View Editor” but rapidly changed it to “ Model View Controller”

Pros and Cons

- Advantages of MVC
 - MVC architecture will separate the user interface from business logic and business logic
 - Components are reusable.
 - Easy to maintain.
 - Different components of the application in MVC can be independently deployed and maintained.
 - This architecture helps to test components independently.
- Disadvantages of MVC
 - The complexity is high.
 - Not suitable for small applications.
 - The inefficiency of data access in view.



The Model

- Most programs are supposed to do work, not just be “another pretty face”
 - but there are some exceptions
 - useful programs existed long before GUIs
- The Model is the part that does the work--it *models* the actual problem being solved
- **The Model should be independent of both the Controller and the View**
 - But it provides services (methods) for them to use
- Independence gives flexibility, robustness

The Controller

- The Controller decides what the model is to do
- Often, the user is put in control by means of a GUI
 - in this case, the GUI and the Controller are often the same
- The Controller and the Model can almost always be separated (what to do versus how to do it)
- The design of the Controller depends on the Model
- The Model should *not* depend on the Controller

The View

- Typically, the user has to be able to see, or *view*, what the program is doing
- The View shows what the Model is doing
 - The View is a passive observer; it should not affect the model
- The Model should be independent of the View, but (but it can provide access methods)
- The View should ***not*** display what the Controller ***thinks*** is happening

Combining Controller and View

- Sometimes the Controller and View are combined, especially in small programs
- Combining the Controller and View is appropriate if they are very interdependent
- The Model should still be independent
- *Never* mix Model code with GUI code!

Learning Objectives

10.1 Definition

10.2 Different understanding of MVC

10.3 Types of MVC

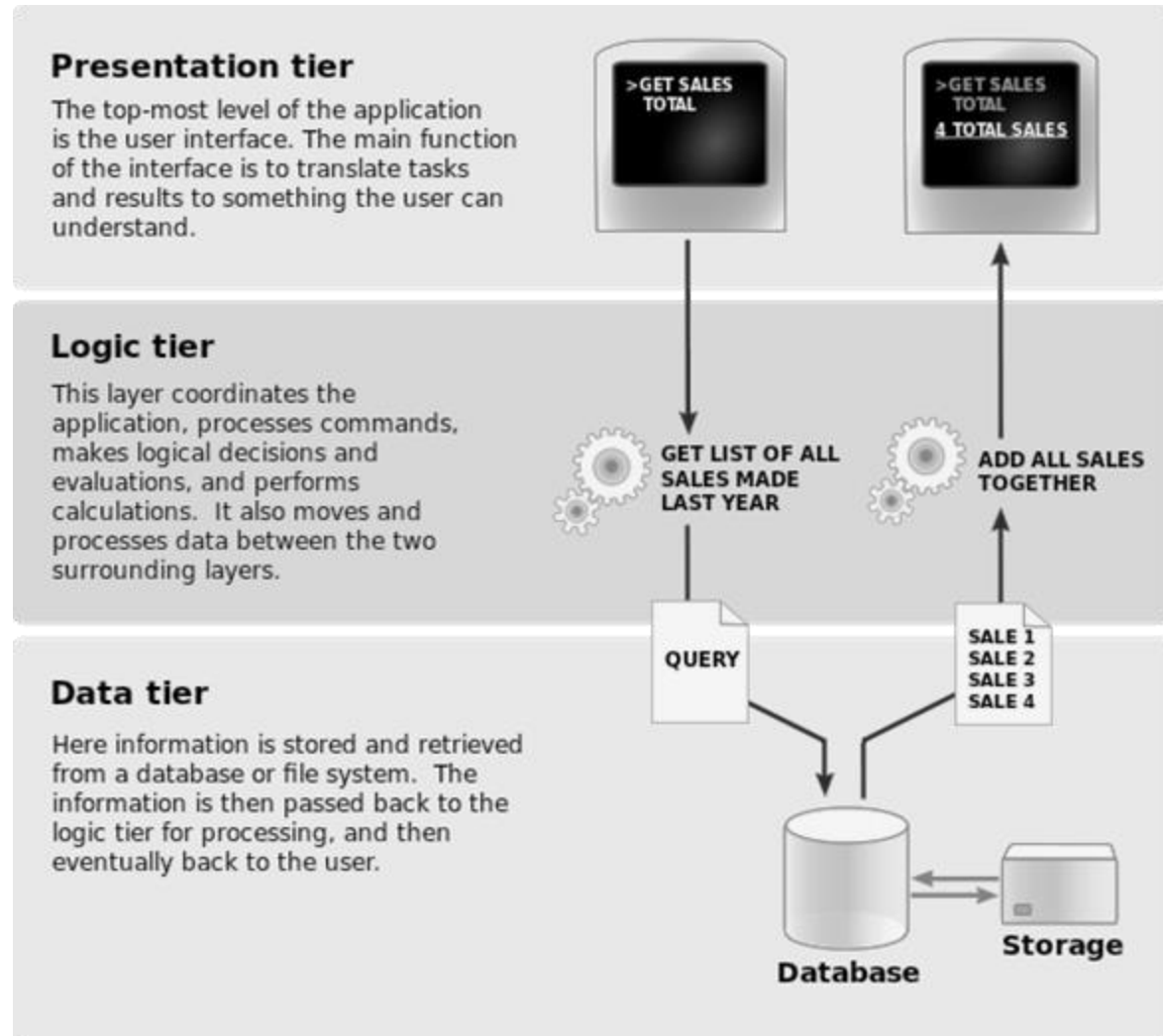
10.4 Example of MVC in different technology

MVC Misunderstood

- MVC is understood by different people in different ways
- It is often misunderstood, but most software developers will say it is important; powerful
- Lets start it right: MVC is a few patterns put together

MVC: Different Understanding (1 of 3)

- Separates representation of information from user interaction
- Promotes:
 - Code Reusability
 - Separation of Concerns

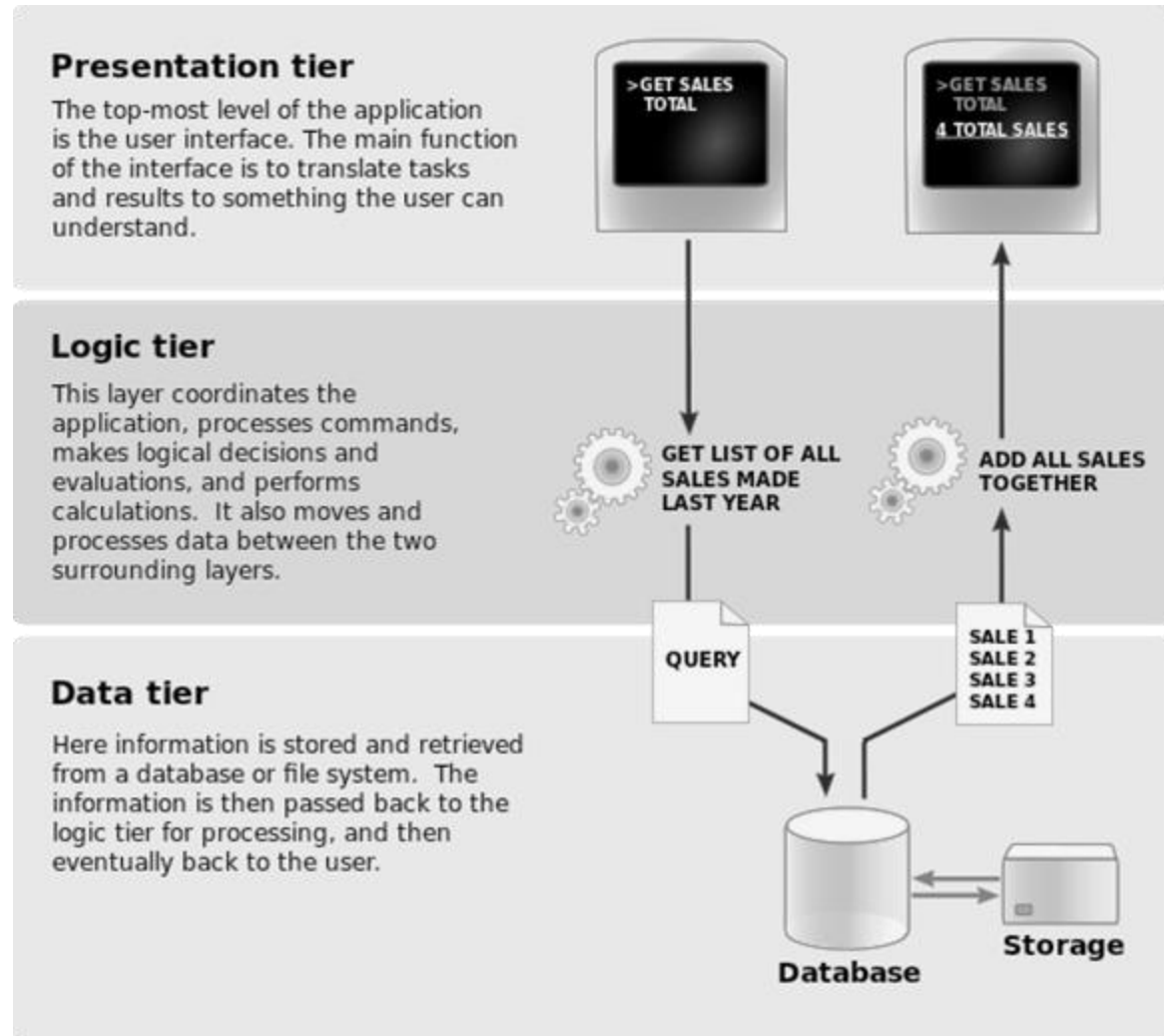


MVC: Different Understanding (2 of 3)

VIEW

CONTROLLER

MODEL

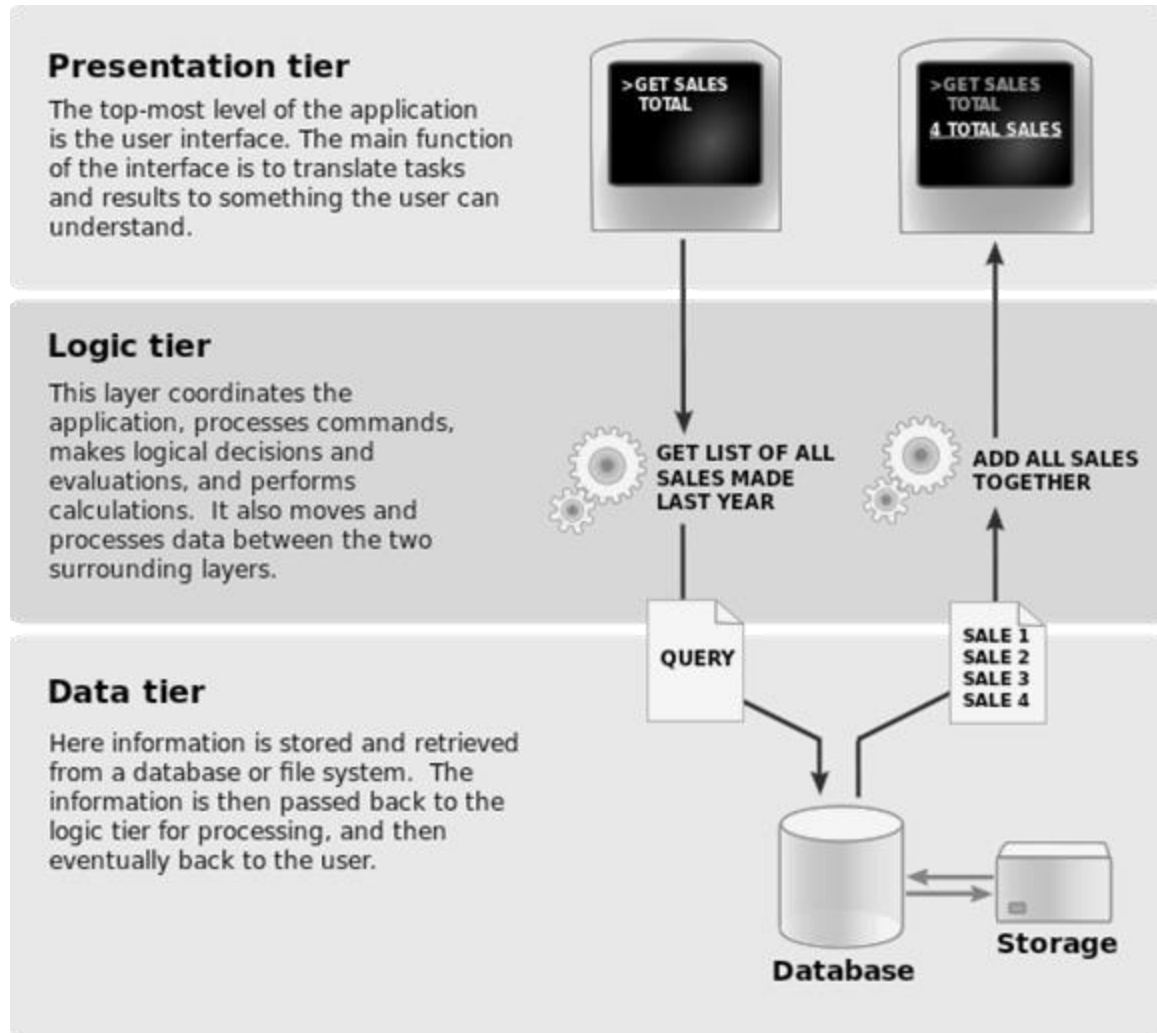


MVC: Different Understanding (3 of 3)

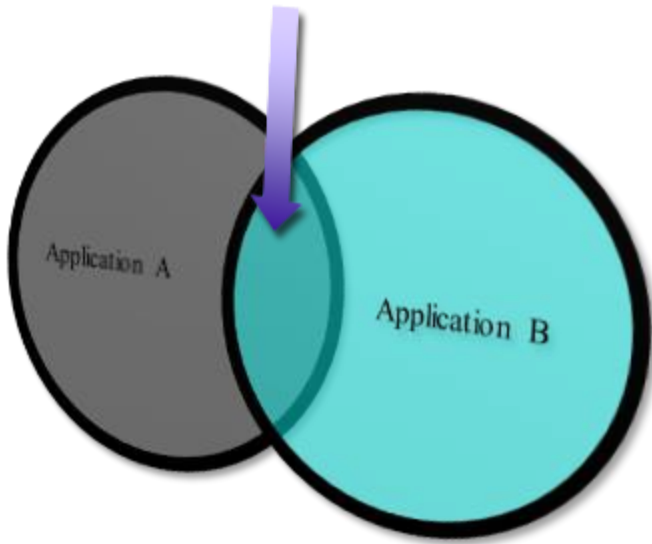
CONTROLLER

VIEW

MODEL

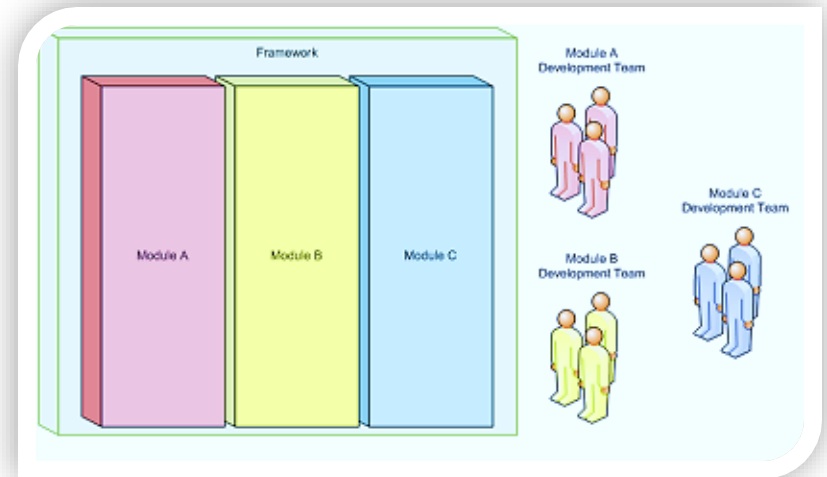


Code Reusability



- Shortens development
- Code Libraries
- Design Patterns
- Frameworks

Separation of Concerns



- Improves code clarity and organization
- Helps troubleshooting by isolating issues
- Allows for multiple teams to develop simultaneously

Learning Objectives

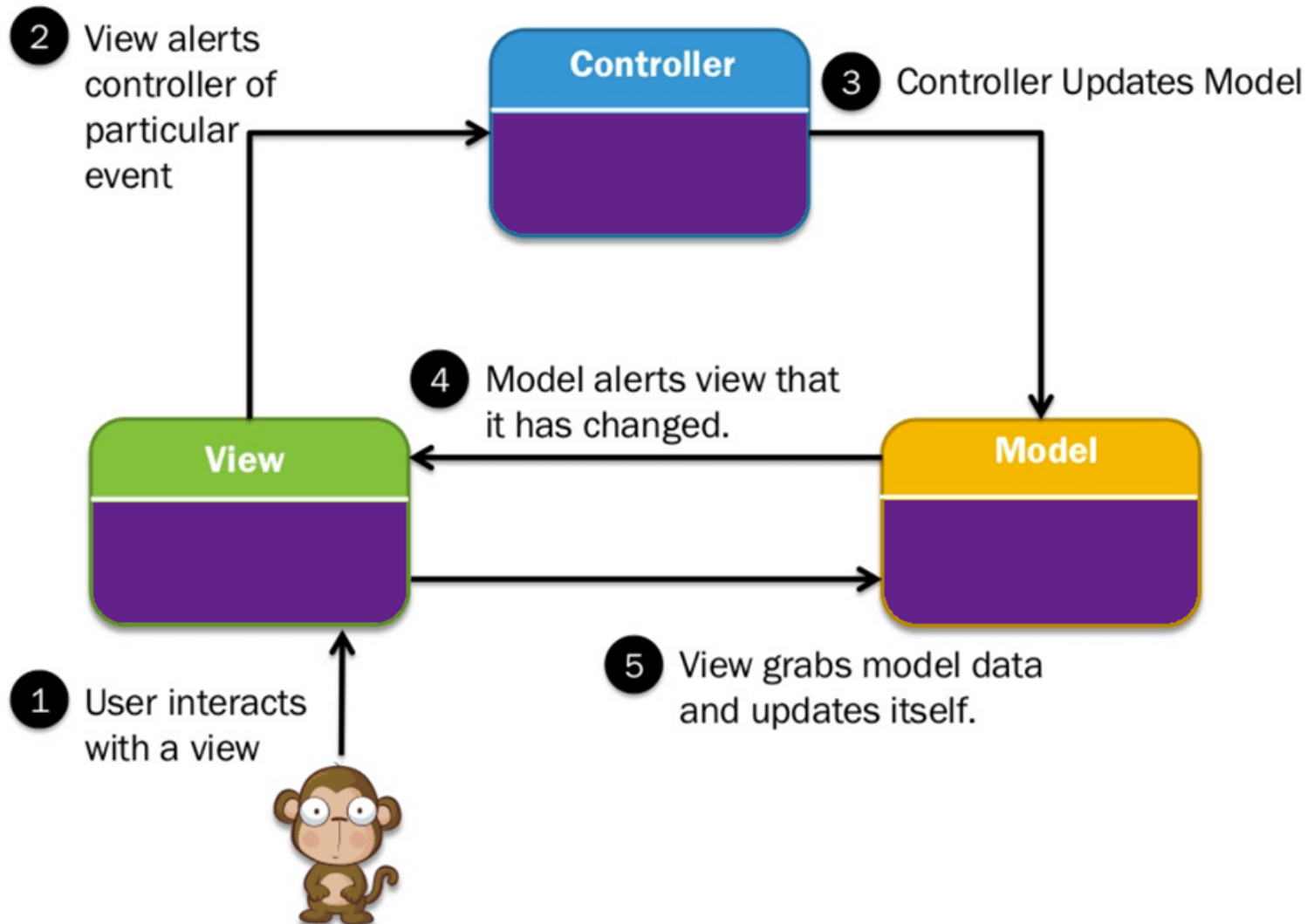
10.1 Definition

10.2 Different understanding of MVC

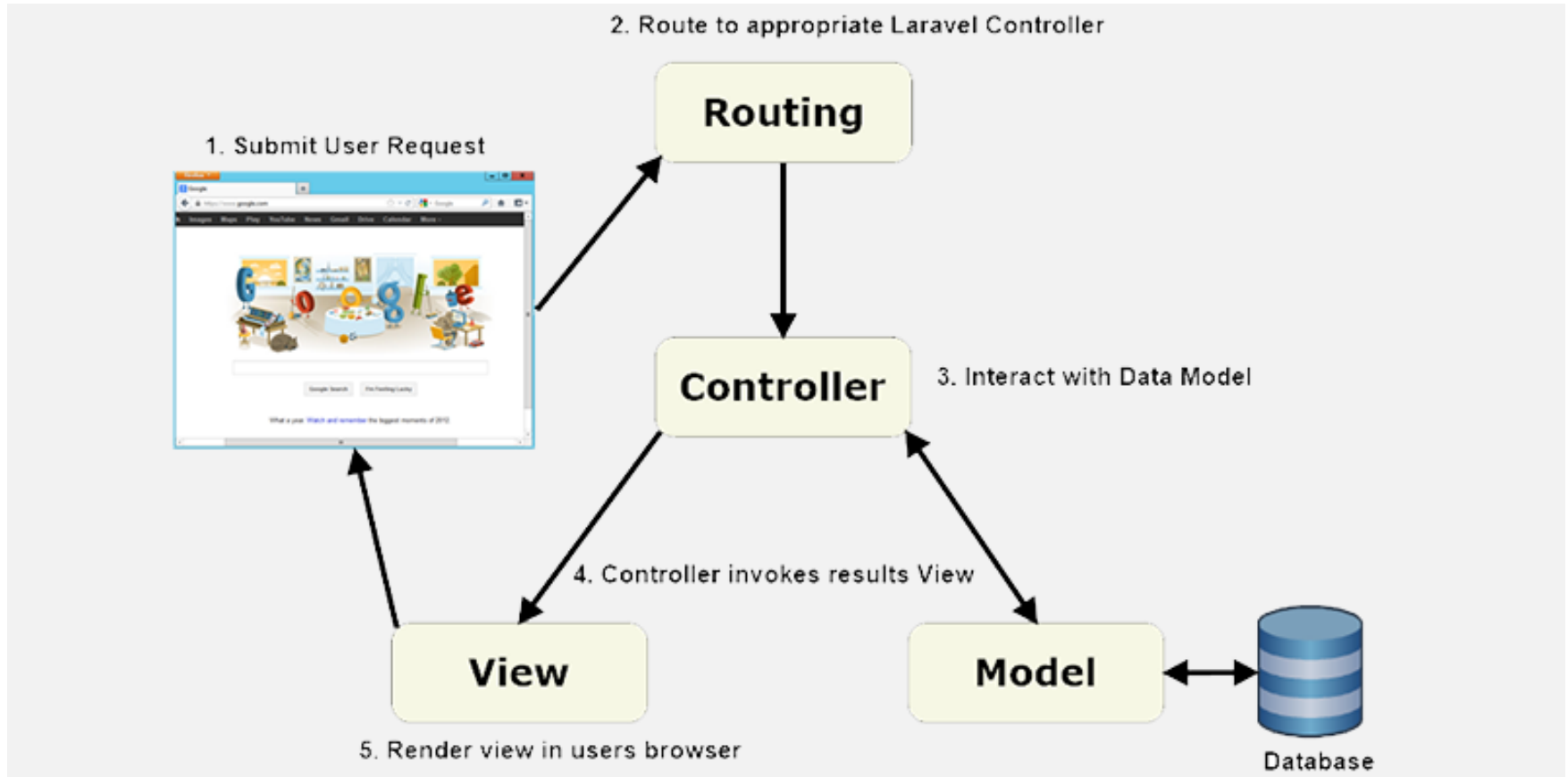
10.3 Types of MVC

10.4 Example of MVC in different technology

How MVC Work: 1st Type

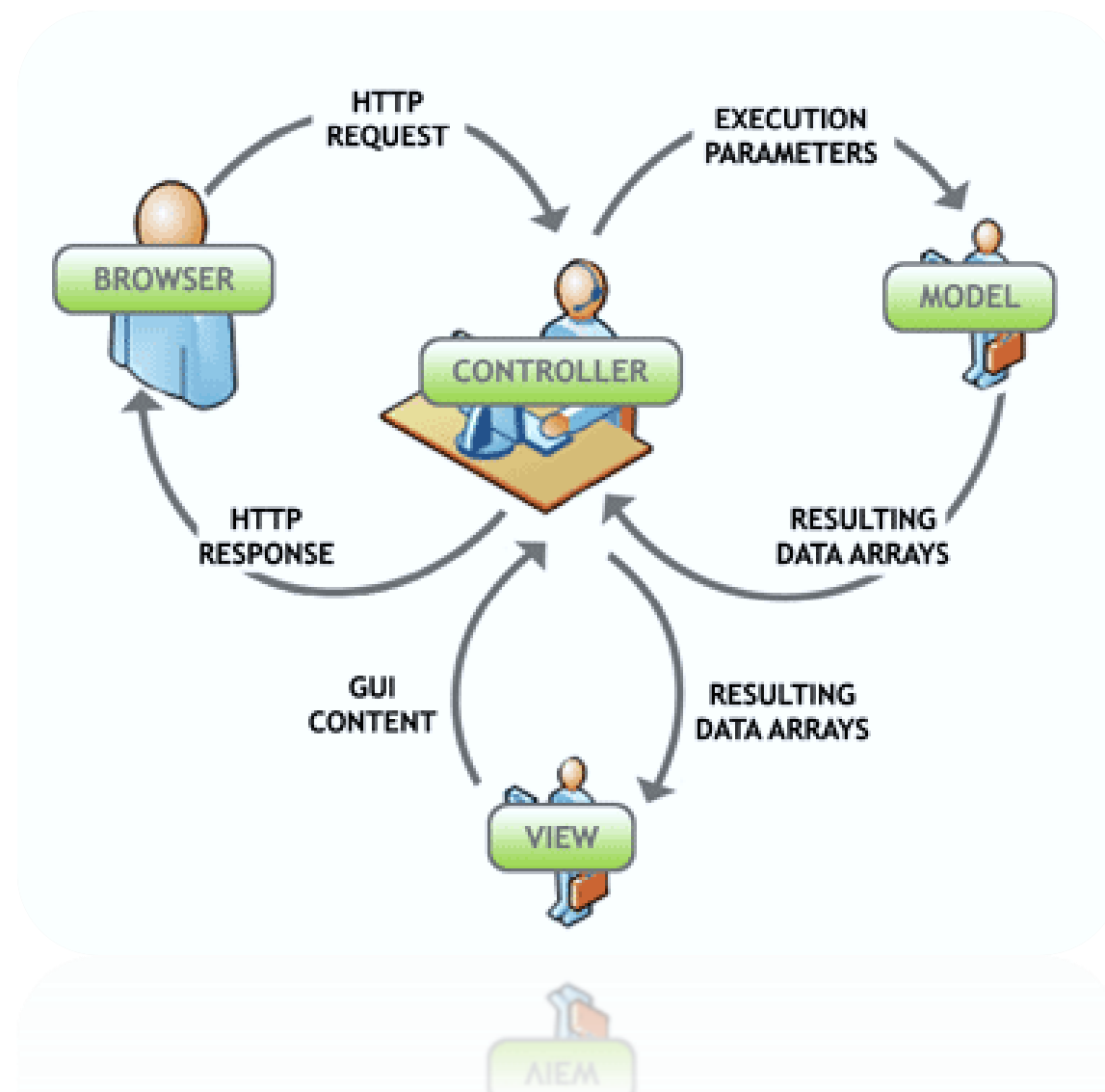


How MVC Work: 2nd Type



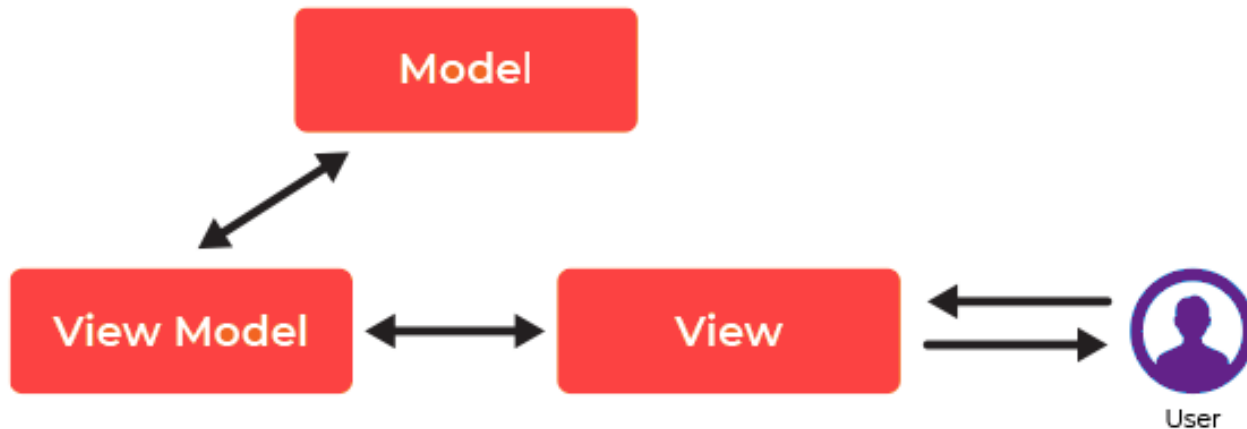
Execution Process

- Controller – Mediates input and commands for the model or view
- Model – Application data, business rules, logic, and functions.
- View – Output and representation of data



How MVC Work: MVVC Model

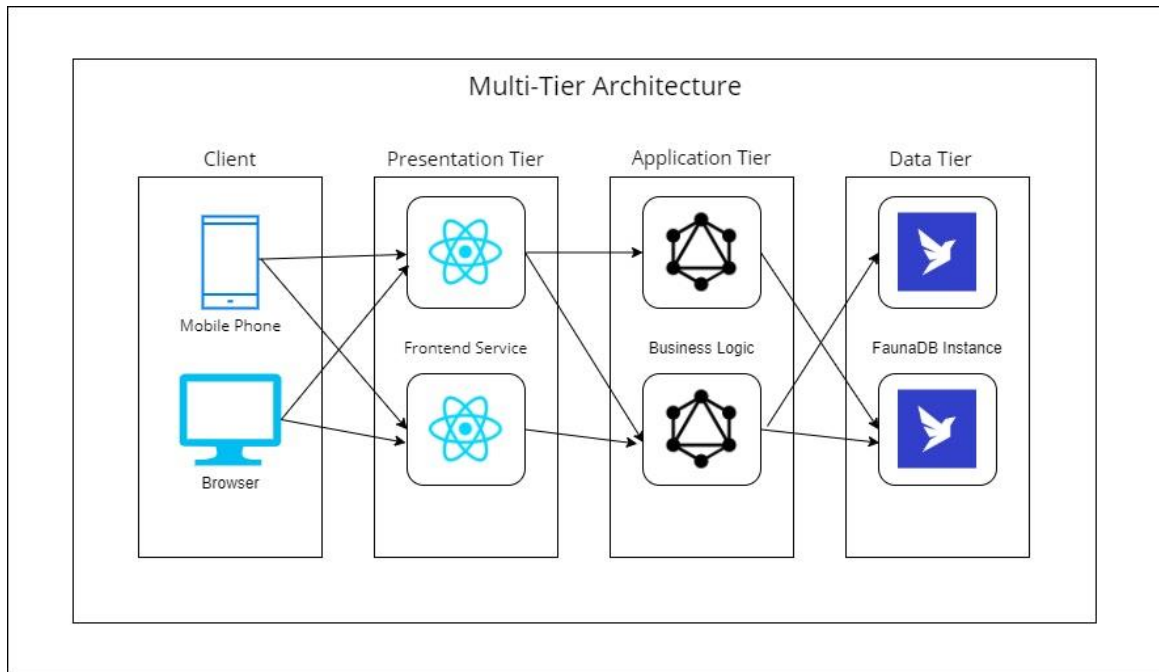
- Instead of the controller of the MVC pattern, or the presenter of the MVP pattern, MVVM has a binder, which automates communication between the view and its bound properties in the view model
- The view model has been described as a state of the data in the model



Improvement of MVC: Multitier architecture (or *n*-tier or multilayer)

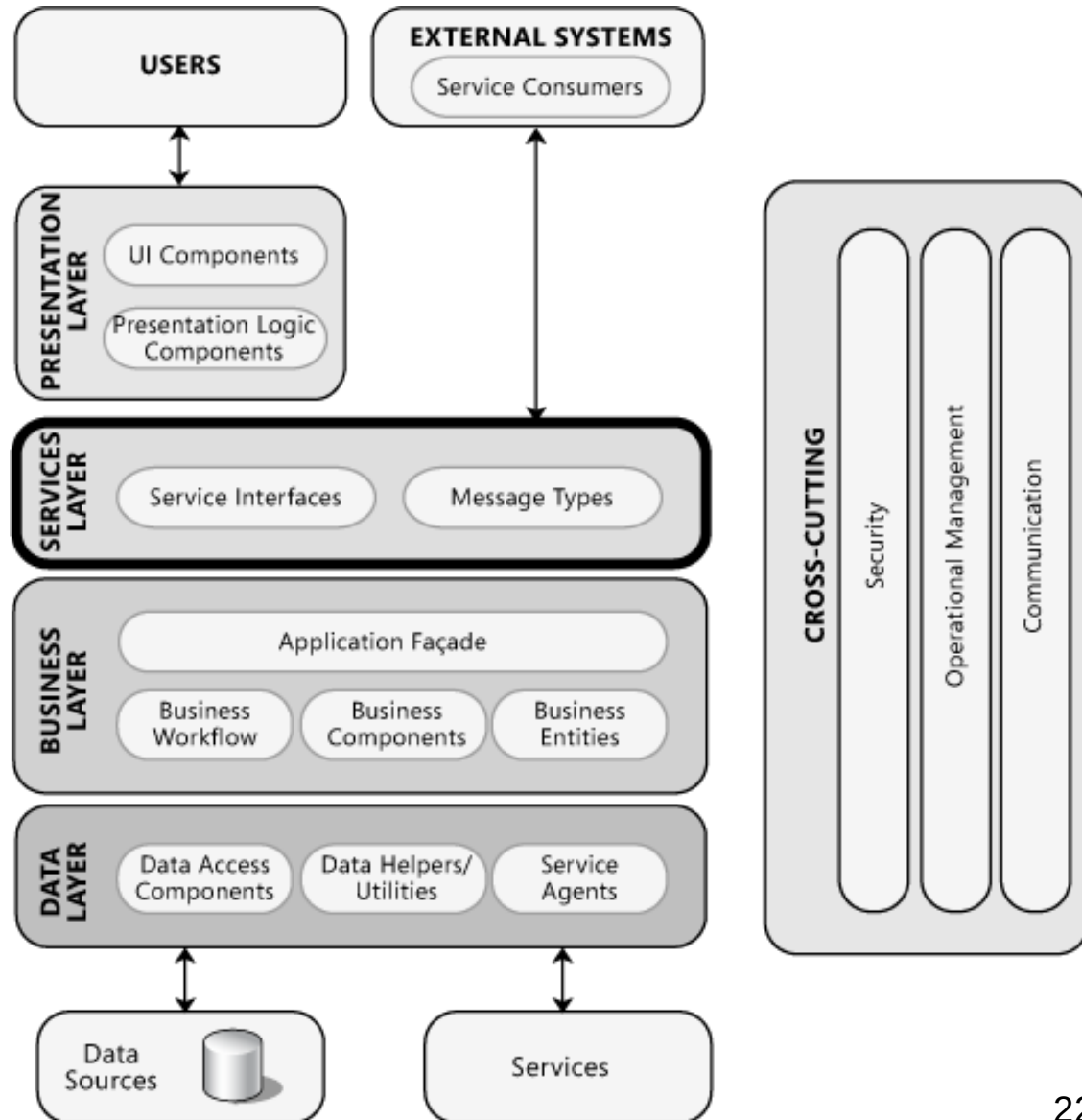
Most common layers:

- Client layer
- Presentation layer
- Application layer
- Business layer
- Integration layer
- Data access layer
- Service layer
- Abstraction layer
- Resource layer



Microsoft Layered Application Architecture (1 of 2)

- Layers are concerned with the logical division of components and functionality
- Layers can be located on different tiers, or they may reside on the same tier



Microsoft Layered Application Architecture (2 of 2)

- Design Steps for a Layered Structure:
 - Step 1 – Choose Your Layering Strategy
 - Step 2 – Determine the Layers You Require
 - Step 3 – Decide How to Distribute Layers and Components
 - Step 4 – Determine If You Need to Collapse Layers
 - Step 5 – Determine Rules for Interaction between Layers
 - Step 6 – Identify Cross Cutting Concerns
 - Step 7 – Define the Interfaces between Layers
 - Step 8 – Choose Your Deployment Strategy
 - Step 9 – Choose Communication Protocols

Learning Objectives

10.1 Definition

10.2 Different understanding of MVC

10.3 Types of MVC

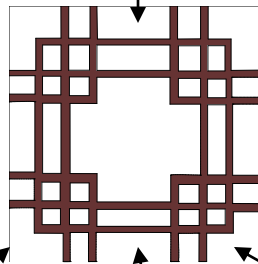
10.4 Example of MVC in different technology

MVC: Example

Model:

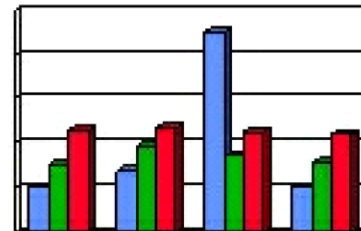
```
float sales[3][4];
```

Controller:

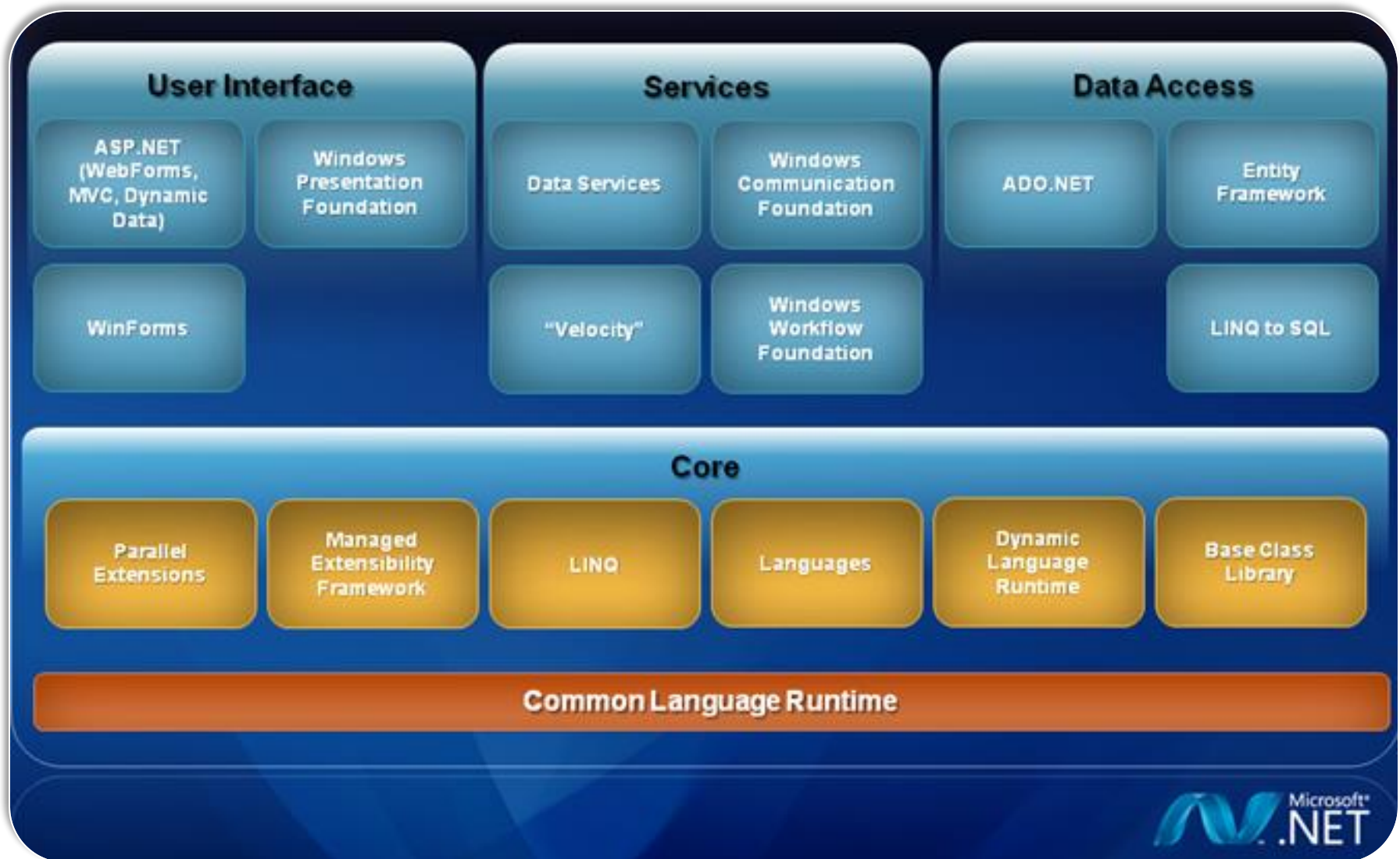


Views:

	1st Qtr	2nd Qtr	3rd Qtr	4th Qtr
East	20.4	27.4	30	20.4
West	30.6	38.6	34.6	31.6
North	45.9	46.9	45	43.9

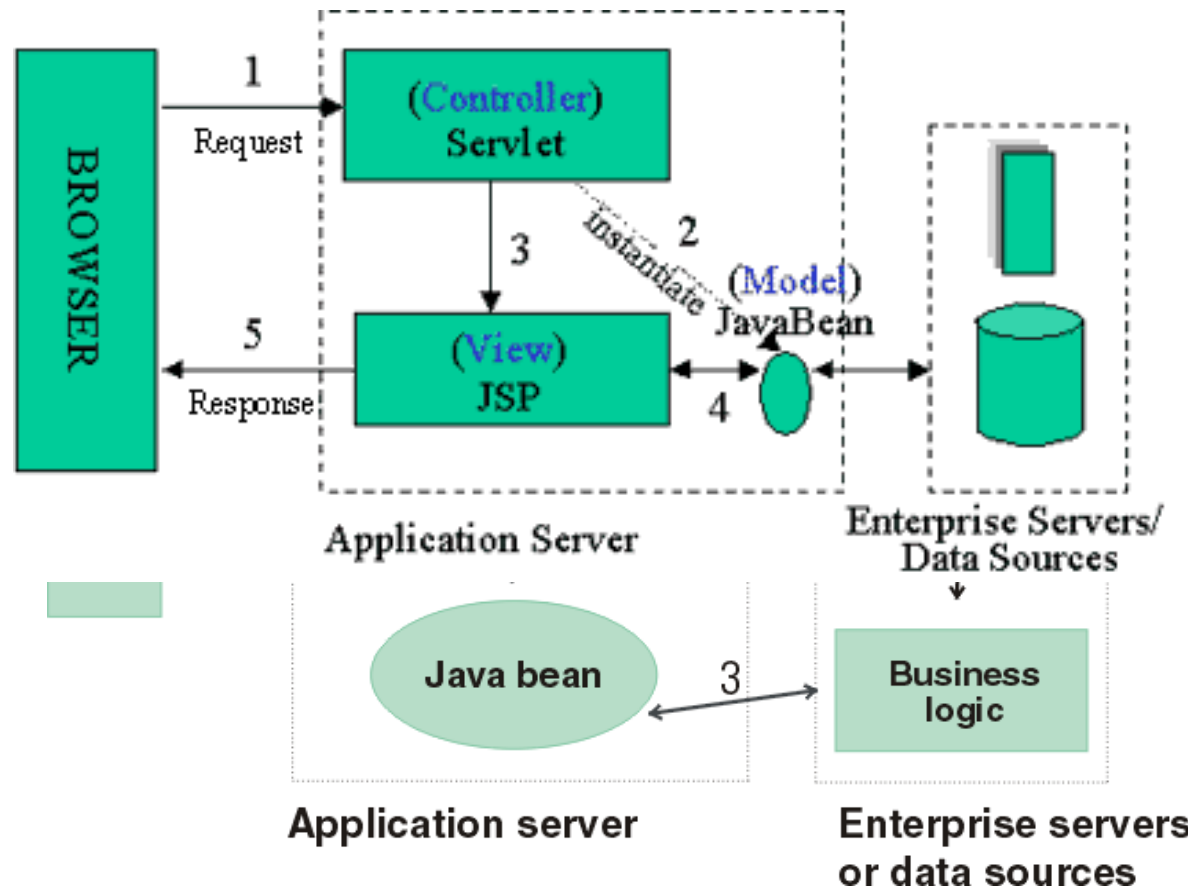


MVC: NET Framework



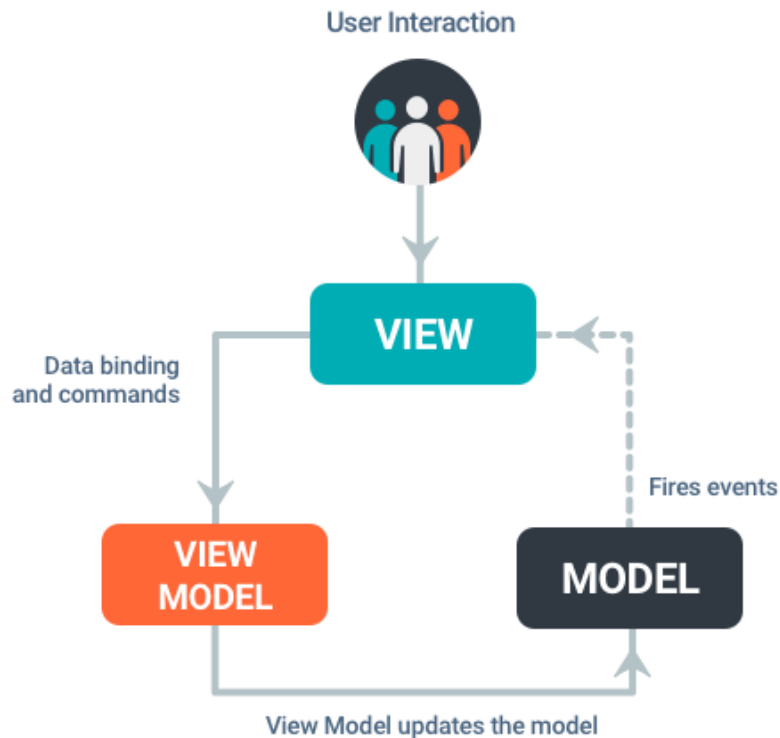
MVC: Java Server Pages

- Model: Enterprise Beans with data in the DBMS
- Controller: Servlets create beans, decide which JSP to return, do the bulk of the processing
- View: The JSPs generated in the presentation layer (the browser)

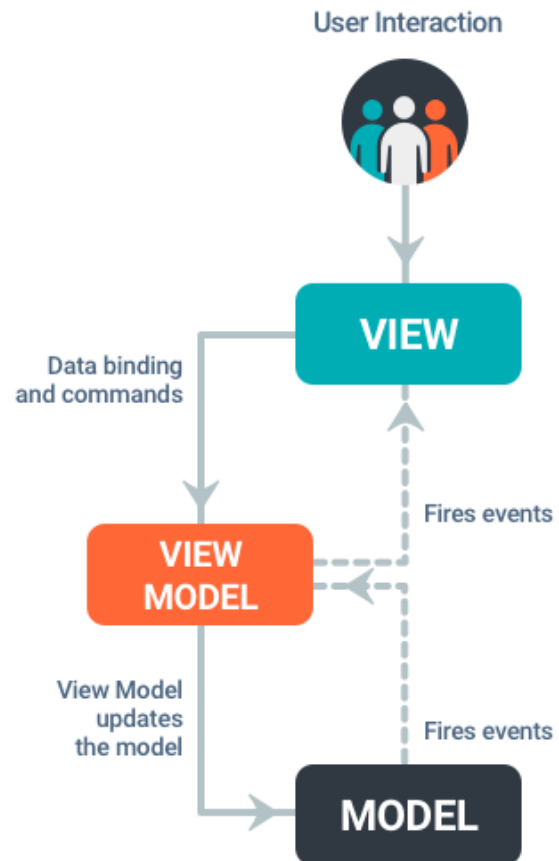


MVC: Android

Model-View-Controller pattern

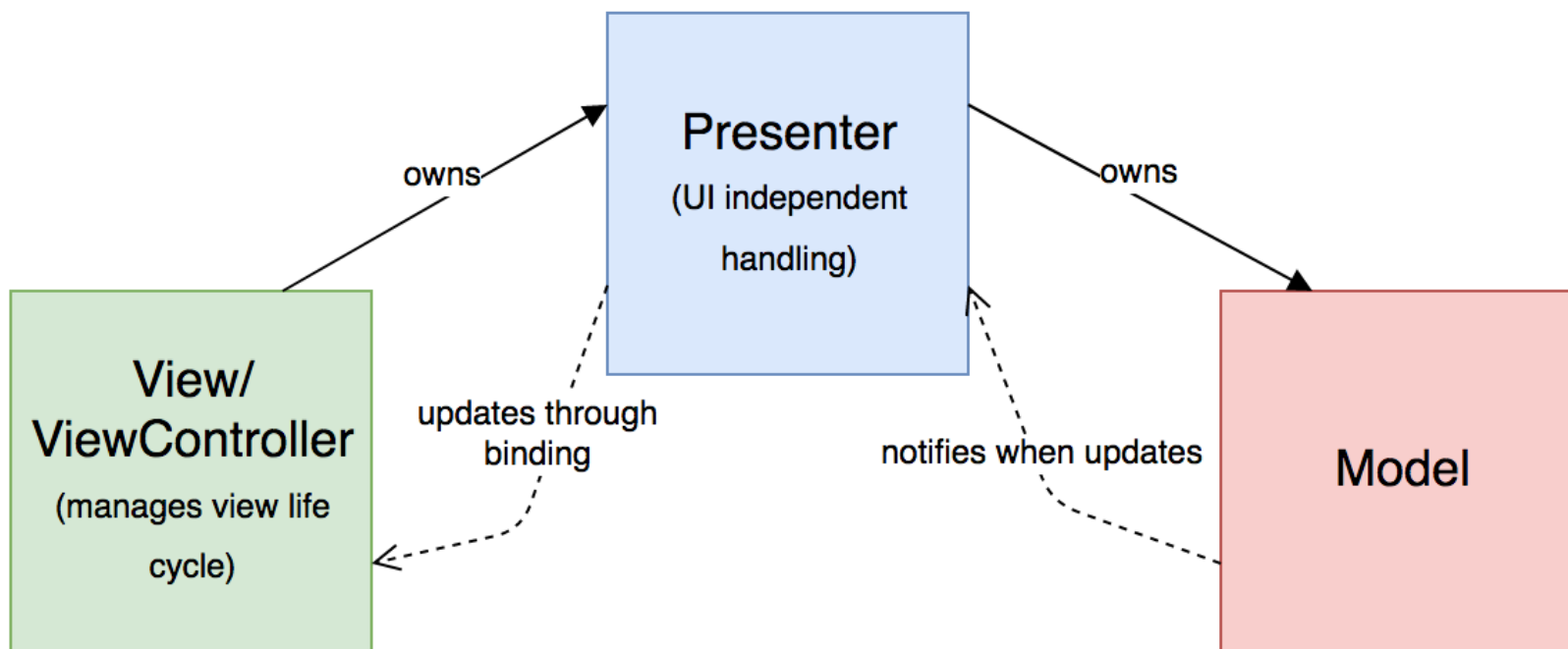


Model-View-ViewModel pattern

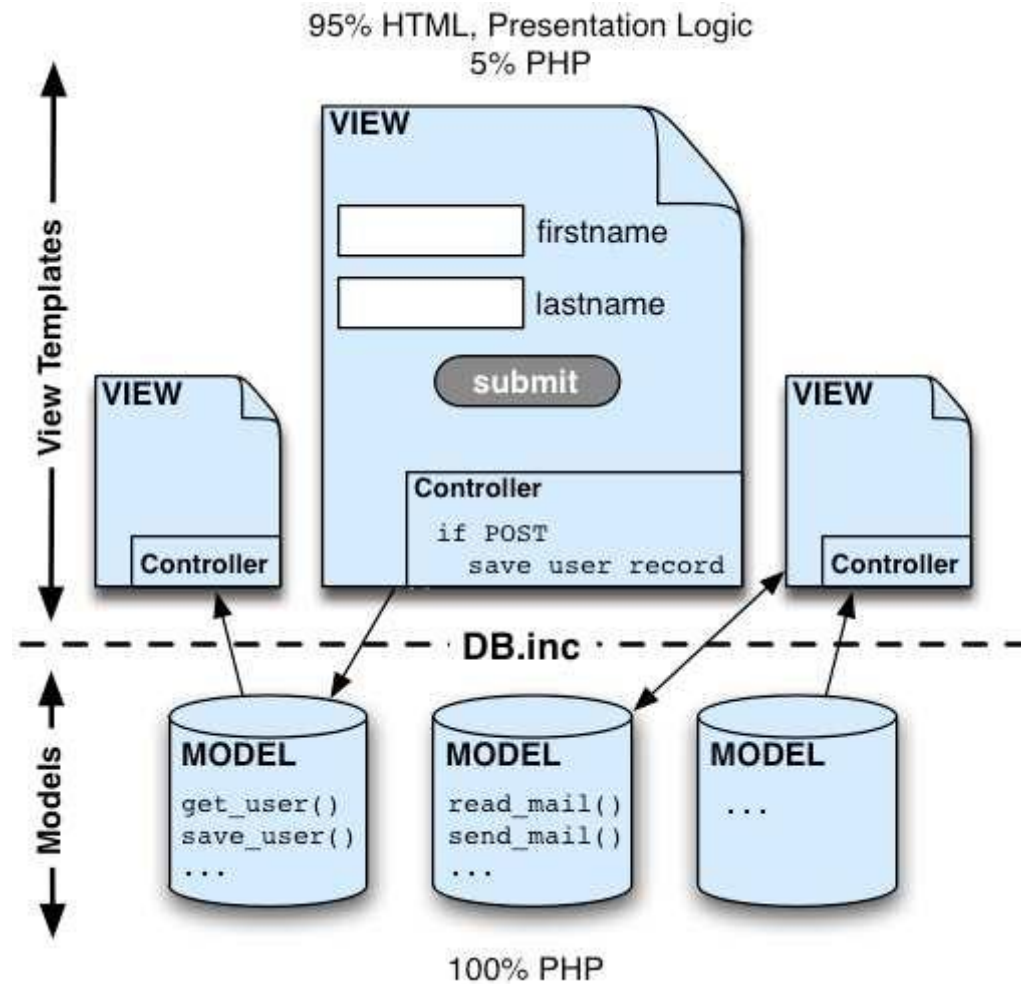
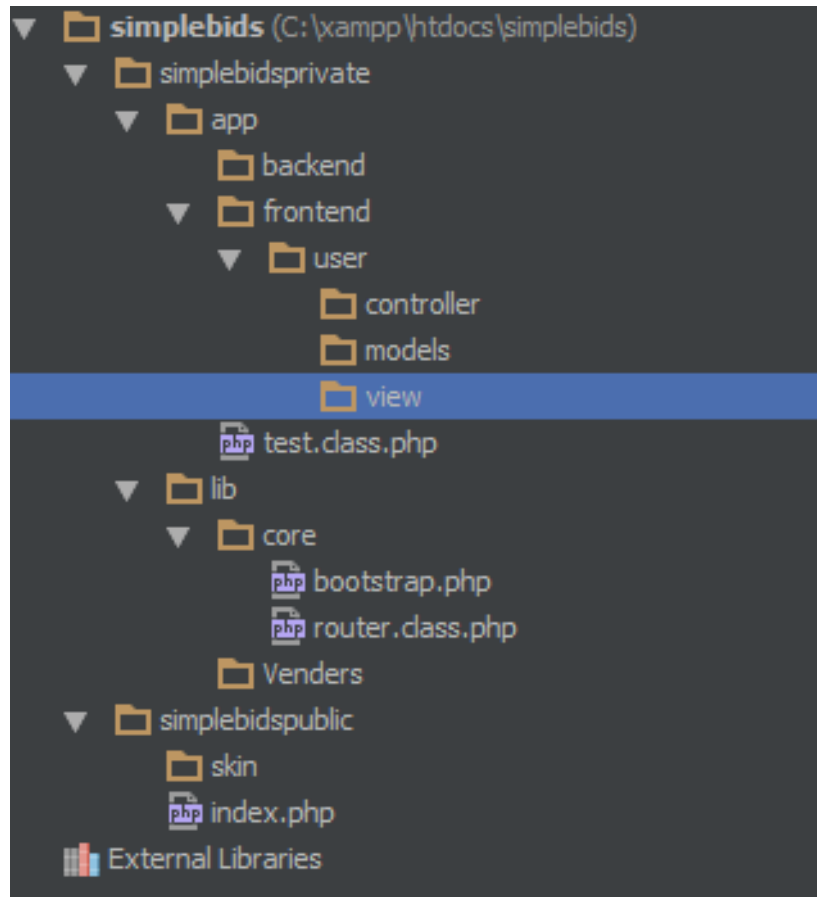


MVC: iOS

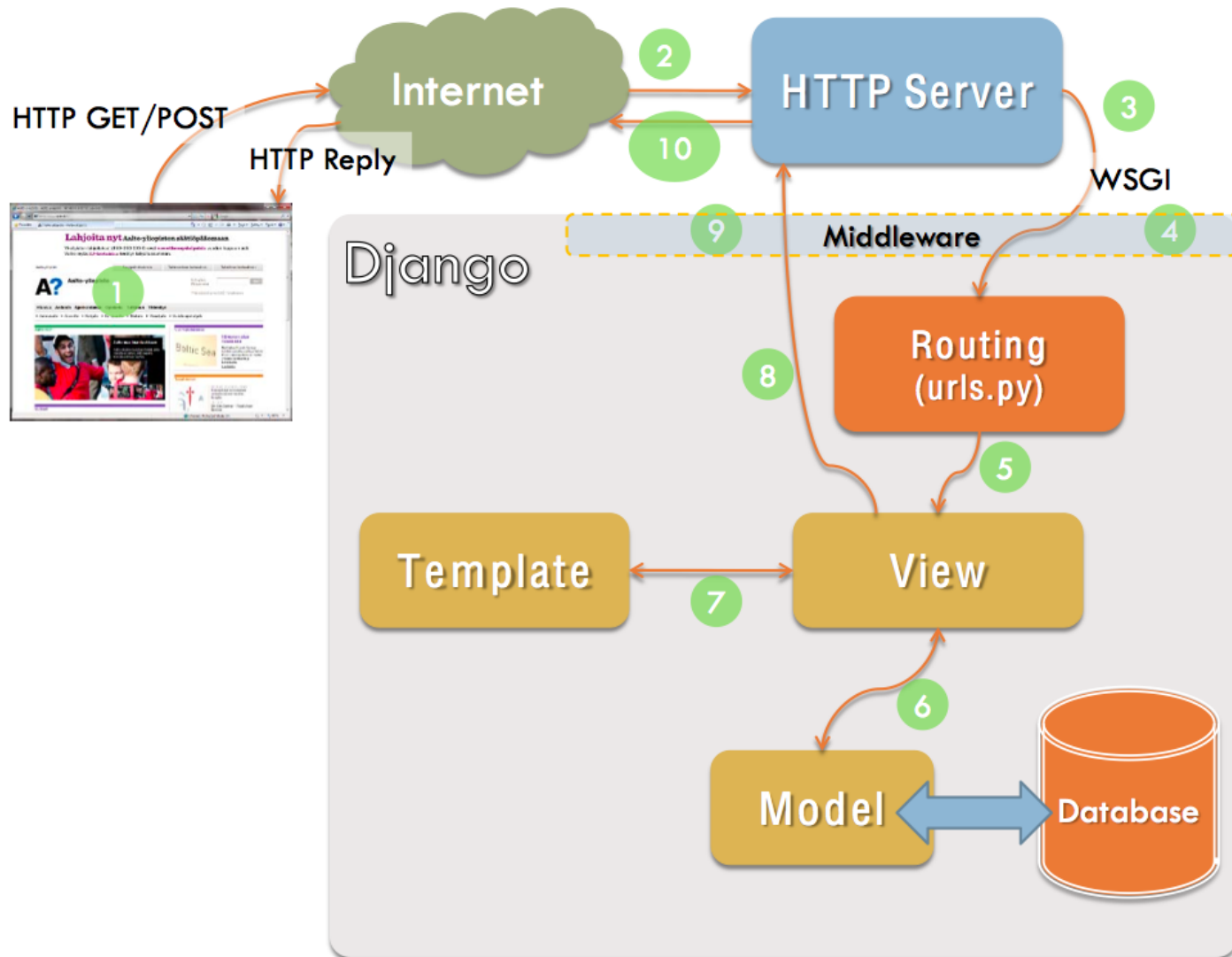
- iOS has already recommended developers to follow MVC
- Apple's MVC is the modified version of the original MVC
- But may fail for complex user interfaces



MVC: PHP

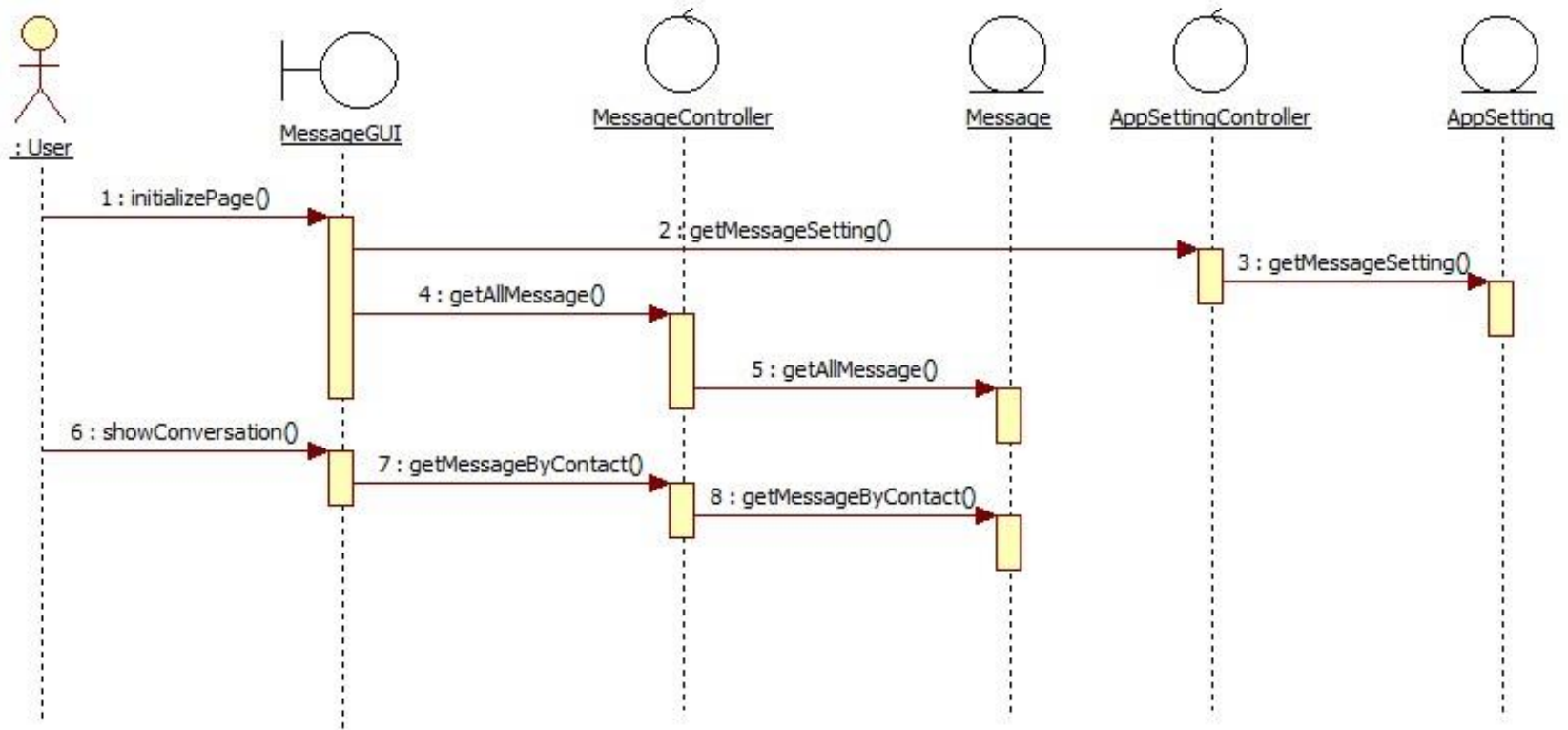


MVC: Python - Django



MVC: RUP

- View: Boundary Controller: Controller Model: Entity



Key points

- A Model does the “business logic”
 - It should be *I/O free*
 - Communication with the Model is via methods
 - This approach gives maximum flexibility in how the model is used
- The Controller organizes the program and provides input (control) to the Model
- The View displays what is going on in the model
 - It should never display what *should* be going on in the model
 - For example, if you ask to save a file, the View shouldn't itself tell you that the file has been saved—it should tell you what the model reports
- Especially in small programs, the Controller and View are often combined

The End

Give someone a program, you frustrate them for a day; teach them how to program, you frustrate them for a lifetime.

— David Leinweber

TIME TO WAKE UP

